Cosa fa una harness

Modulo 4 · Lezione 1



Stesso modello, harness diverse

Terminal-Bench 2.0: 89 compiti da svolgere in un terminale. Lo stesso modello, **Claude Opus 4.6**, dentro tre programmi diversi.



> Il modello è lo stesso: cambia il codice che gli sta intorno. Quel codice si chiama **harness**.

Claude Code, uno dei coding agent che userete, qui è ultima: non è fatta per questo benchmark. Ci torniamo alla fine.

Perché ne parliamo

1. La harness **cambia il risultato**, a parità di modello.

2. Ne avete **già scritta una**, di base: `chiedi()`. Oggi guardiamo dentro harness vere, **riga per riga**.

3. Nelle prossime lezioni lavorate con i **coding agent**, cioè con harness molto più grandi.

la domanda uscita dopo Modulo 3 · Lezione 10

«Perché l'Agents SDK e non `aikit`?»

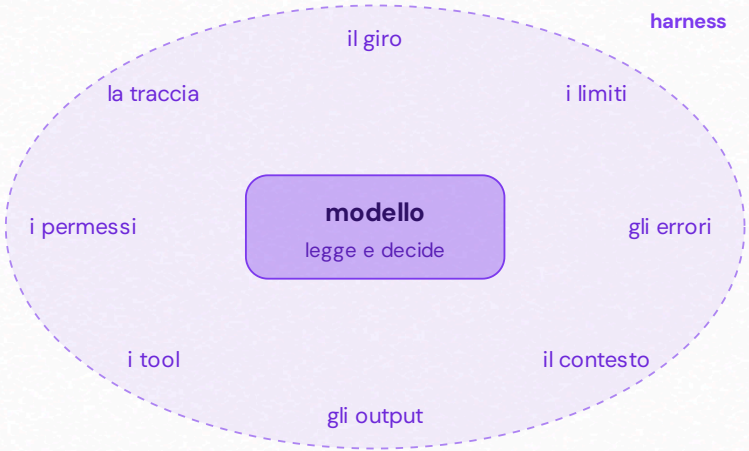
La risposta è in fondo alla lezione.

Agenda

	Blocco	Tempo
	Stesso modello, harness diverse	5′
	Cos'è e quanto conta	7′
	Una harness intera in Python	13′
	Cosa riceve il modello alla prima chiamata	12′
	I tool, da vicino	13′
	Cosa aggiunge la harness durante il giro	10′
	Leggere una trace · `scripts/dataset/trace-claude-code.md` + `scripts/working/scheda.md`	20′ + 5′
	Il contesto quando si riempie	15′
	Errori, cicli, fine del giro	8′
	Permessi e sandbox	5′
	Perché harness esistenti e coding agent	7′

Niente codice da scrivere: si legge codice di harness vere.

Agente = modello + harness



Il **modello** legge un input e scrive un output: testo, o la richiesta di un tool. **Decide lui** cosa chiedere.

Tutto il resto lo fa la **harness**: gli otto pezzi intorno.

> «everything in an AI agent except the model itself»
> tutto quello che c'è in un agente, tranne il modello
> martinfowler.com, 2026

Due parole: una **chiamata** è un invio della harness al modello (il turno di Modulo 3 · Lezione 10). Una **richiesta** è un messaggio dell'utente con tutte le chiamate che ne seguono (lo scambio di Modulo 3 · Lezione 11).

Senza harness e con una harness

La domanda di Modulo 3 · Lezione 8, a gpt-5.6-luna, cinque volte: «Quanti ordini ha fatto Giulia Moretti?»

MODELLO DA SOLO

0 su 5

→ Non dispongo di dati sufficienti per sapere quanti ordini abbia fatto Giulia Moretti.

le altre quattro: «non ho accesso ai dati»

NEL GIRO DI MODULO 3 · LEZIONE 8, CON query_db

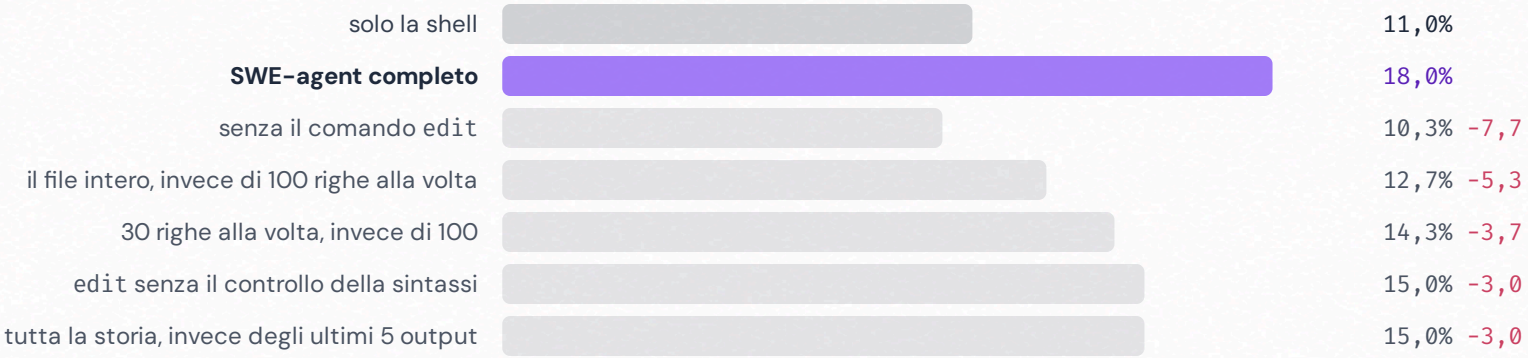
5 su 5

chiamata 1 → query_db(…)
chiamata 2 → Giulia Moretti ha effettuato 13 ordini.

> Il modello da solo scrive testo. Per **agire** (leggere un database, un file, lanciare un comando) gli serve una harness.

Ogni scelta della harness si misura

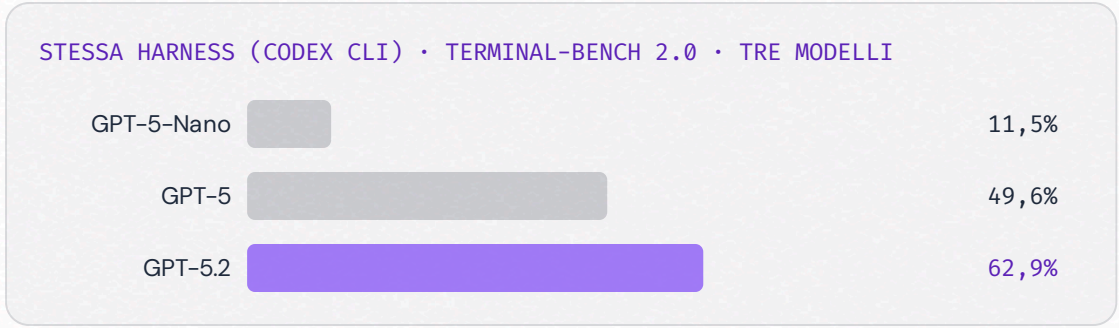
SWE-agent (2024): correggere bug veri in repository GitHub, sempre con GPT-4 Turbo. Prima la sola shell, poi l'interfaccia completa, poi un pezzo tolto alla volta. Le rivediamo più avanti, con il codice.



LangChain, stesso modello (gpt-5.2-codex), solo la harness migliorata: da **52,8%** a **66,5%** su Terminal-Bench 2.0.

Codice: **SWE-agent** config/sweagent_0_7/07.yaml, la configurazione del paper

Il modello conta di più



cambiare modello

fino a 51 punti

cambiare harness, a modello fisso: negli esempi di questa lezione

da 3 a 41 punti

41: Aider, solo il formato delle modifiche (lo vediamo nei tool)

«model selection is usually more important than agent scaffold»

di solito la scelta del modello conta più della harness («scaffold» è un altro nome per harness)

gli autori di Terminal-Bench, 2026

> Prima si sceglie il modello, poi si cura la harness.

Le harness della lezione

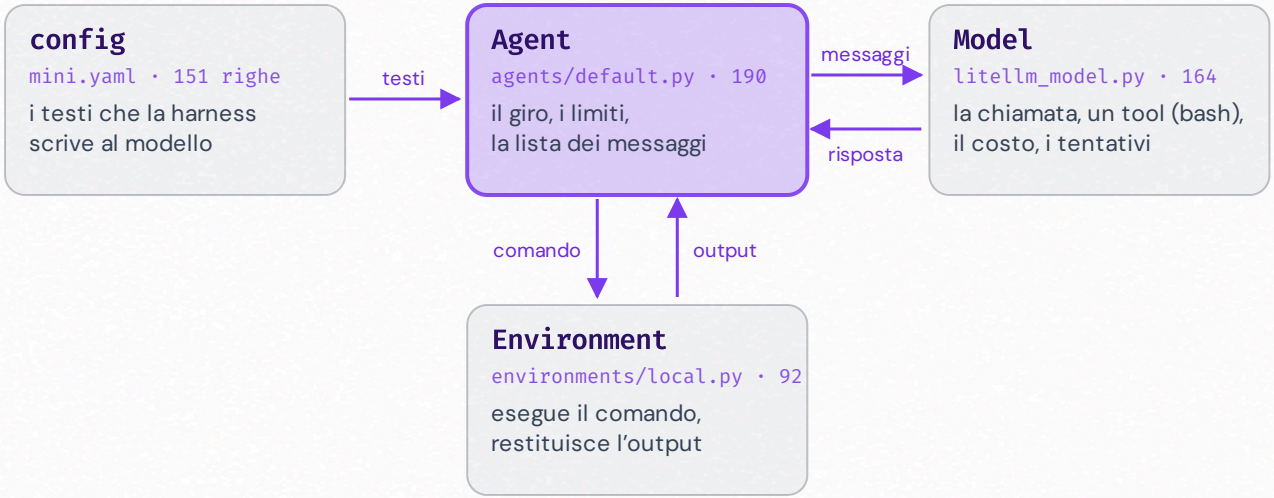
	Cos'è	
mini-swe-agent	la harness più piccola degli autori di SWE-agent: un solo tool, la shell	Python
SWE-agent	la harness del paper del 2024. SWE-bench è il benchmark, non una harness	Python
Aider	un assistente di programmazione da terminale: niente tool, le modifiche nel testo	Python
OpenHands	una piattaforma open source di agenti per il codice; qui il suo SDK	Python
smolagents	la libreria di agenti di Hugging Face: l'azione è codice Python	Python
Codex	il coding agent di OpenAI	Rust
Claude Code	il coding agent di Anthropic, non open source; è quello della trace	—
tool di riferimento di Anthropic	il codice d'esempio che Anthropic pubblica per i suoi tool (editor di file, bash)	Python
Terminus 2	la harness di riferimento di Terminal-Bench: una sola shell	Python
Meta-Harness	una harness ottimizzata in automatico, da un altro programma, su Terminal-Bench	Python

`IL PROGETTO DEGLI ESEMPI E DELL'ESERCIZIO` · Valtesa: un piccolo programma Python che controlla 1.500 note spese contro i massimali aziendali. Ha un bug: la nota 1043, una cena a Milano da 52 €, risulta fuori massimale. Da Modulo 4 · Lezione 2 ci lavorate voi con il coding agent, sulla versione con il bug già corretto.



mini-swe-agent: com’è fatta

Una harness intera, in Python: tre classi e un file di testi.



76,8%

di SWE-bench Verified con Claude Opus 4.5. Il miglior risultato, con qualunque harness: 79,2%.

Il pacchetto intero: 5.080 righe.

Una **corsa**: mini-swe-agent che lavora su un compito, dall'inizio alla fine. Dopo l'occhiata ai file, quattro slide sono il suo codice e i suoi testi, tagliati; i commenti in italiano sono nostri.

Codice: **mini-swe-agent** agents/default.py · models/litellm_model.py · environments/local.py · config/mini.yaml

Il sorgente di mini-swe-agent

GUARDIAMO INSIEME — il docente apre i file, voi potete seguire

`SWE-agent/mini-swe-agent · versione 2.4.6 · 3 minuti`

poi il docente riprende le slide

`src/minisweagent/agents/default.py`	190 righe	**Agent**: il giro, i limiti, la lista dei messaggi
`src/minisweagent/models/litellm_model.py`	164 righe	**Model**: la chiamata al modello, il tool bash
`src/minisweagent/environments/local.py`	92 righe	**Environment**: esegue il comando
`src/minisweagent/config/mini.yaml`	151 righe	la config: i testi che la harness scrive al modello

Repository al commit usato nelle slide: `github.com/SWE-agent/mini-swe-agent`

Il giro di mini-swe-agent

```python
def run(self, task):
    self.messages = []
    self.add_messages(system_msg, instance_msg)   # system + il compito
    while True:
        try:
            self.step()                           # query() + execute_actions()
        except FormatError as e:
            self.add_messages(*e.messages)        # solo l'errore, non la risposta
        except InterruptAgentFlow as e:
            self.add_messages(*e.messages)        # Submitted, LimitsExceeded
        finally:
            self.save(self.config.output_path)    # la traiettoria, su disco
        if self.messages[-1].get("role") == "exit":
            break

def query(self):
    if 0 < self.config.step_limit <= self.n_calls or \
       0 < self.config.cost_limit <= self.cost:
        raise LimitsExceeded({"role": "exit", "content": "LimitsExceeded"})
    self.n_calls += 1
    message = self.model.query(self.messages)
    self.cost += message.get("extra", {}).get("cost", 0.0)
    self.add_messages(message)
    return message
```

Il giro si ferma quando l'ultimo messaggio ha ruolo `exit`. Lo scrive la harness, e al modello **non arriva mai**.

I limiti si controllano **prima** della chiamata: `step_limit` conta le chiamate, come `MAX_CHIAMATE`; `cost_limit` i dollari (3 $ nella config).

Al limite, al modello non si dice niente: la corsa finisce. Un limite a 0 vuol dire nessun tetto.

Dopo ogni passo la storia viene salvata su file: è la sua **traccia**.

Codice: **mini-swe-agent** agents/default.py run() e query() · models/litellm_model.py dove nasce FormatError · config/mini.yaml, i limiti



Il tool di mini-swe-agent: un processo con un timeout

```python
def _run(command, cwd, env, timeout):
    process = subprocess.Popen(
        command, shell=True, text=True, cwd=cwd, env=env,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,          # errori e output insieme
        start_new_session=True,            # un gruppo di processi suo
    )
    try:
        stdout, _ = process.communicate(timeout=timeout)   # 30 s
    except subprocess.TimeoutExpired:
        os.killpg(process.pid, signal.SIGKILL)   # uccide anche i figli
        ...

def _check_finished(self, output):
    lines = output.get("output", "").lstrip().splitlines(keepends=True)
    if lines and lines[0].strip() == "COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT" \
       and output["returncode"] == 0:
        raise Submitted({"role": "exit", "content": "".join(lines[1:])})
```

Ogni comando è un **processo nuovo**: riparte dalla cartella iniziale, un cd non resta. Il prompt lo dice al modello: *«Every action is executed in a new subshell.»*

Dopo 30 secondi il comando viene ucciso, con tutti i processi che ha lanciato. Al modello torna *«Command '…' timed out after 30 seconds»*.

«Ho finito» è una **stringa**: il modello deve eseguire echo `COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`. Se la risposta non chiama nessun tool, per mini è un errore.

Codice: **mini-swe-agent** environments/local.py, _run() · _check_finished() · config/mini.yaml, la subshell nuova



Cosa torna al modello, in mini-swe-agent

DOPO OGNI COMANDO · IL TEMPLATE JINJA, IN mini.yaml

```
{%- if output.output | length < 10000 -%}
{ "returncode": {{ output.returncode }},
  "output": {{ output.output | tojson }} }
{%- else -%}
{ "returncode": {{ output.returncode }},
  "output_head": {{ output.output[:5000] | tojson }},
  "output_tail": {{ output.output[-5000:] | tojson }},
  "elided_chars": {{ output.output | length - 10000 }},
  "warning": "Output too long." }
{%- endif -%}
```

Testo con i segnaposto tra {{ }} e le condizioni tra {% %}. Sul progetto Valtesa, `python rimborsi.py` stampa **108.707** caratteri: il modello ne vede **10.000**, i primi e gli ultimi. L'ultima riga, il totale, c'è.

SE IL MODELLO NON CHIAMA bash · ERRORE DI FORMATO

```
Tool call error:

<error>
No tool calls found in the response. Every response MUST include
at least one tool call.
</error>

Here is general guidance on how to submit correct toolcalls:

Every response needs to use the 'bash' tool at least once to
execute commands.
…
If you want to end the task, please issue the following command:
`echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`
```

Nella storia entra **solo l'errore**: la risposta sbagliata no. Tre errori di fila, e la corsa finisce.

Codice: **mini-swe-agent** config/mini.yaml, observation_template · format_error_template · actions_toolcall.py, nessun tool · agents/default.py, tre errori di fila



I messaggi di una corsa

system	«You are a helpful assistant that can interact with a computer.»	12
user	il compito · 47 + workflow in 6 passi, regole, come si chiude, esempi di sed	702
tools	lo schema di bash, a ogni chiamata	64
assistant	testo + `bash("grep -n massimale rimborsi.py")`	
tool	`{"returncode": 0, "output": "…"}`	
assistant	solo testo, nessun tool: non entra nella storia	
user	«Tool call error: …»	
	… comandi e output …	
assistant	`bash("echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT")`	
exit	fine della corsa: il modello non lo legge	

- scritto dall'utente
- scritto dal modello
- scritto dalla harness

A destra, i token.

Il compito sono 47 token su circa 780. Il resto lo scrive la harness.

- il modello non lo rilegge

Ricostruita dal codice e dai template, non da una corsa.

Codice: **mini-swe-agent** config/mini.yaml, system e istruzioni · actions_toolcall.py, lo schema di bash · litellm_model.py, extra tolto prima dell'invio



mini-swe-agent e la vostra chiedi()

	Voi · `chiedi()`	mini-swe-agent
I tool	`calcolatrice`, `data_oggi`, `query_db`, …	uno solo: `bash`
Quando finisce	una risposta senza tool	solo con `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`; una risposta senza tool è un errore
Il tetto	`MAX_CHIAMATE = 6`, prima della chiamata	`step_limit` e `cost_limit`, prima della chiamata
Al tetto	un messaggio all'utente	un messaggio `exit`; al modello niente
L'errore di un tool	«Errore: …» come testo, in `query_db`	returncode e output
Gli output lunghi	`LIMITE = 50` righe	5.000 caratteri in testa e 5.000 in coda
L'API che non risponde	`client = OpenAI()`: 2 tentativi in più	10 tentativi, attese da 4 a 60 s

> La struttura è la stessa. La differenza più grande è **come decide di fermarsi**: mini-swe-agent vuole che il modello dichiari la fine; il vostro giro, e anche Claude Code, si fermano quando il modello smette di chiedere tool (slide 46).

Codice: **mini-swe-agent** agents/default.py, query() · actions_toolcall.py, risposta senza tool · models/utils/retry.py **client openai 3.19.2** _constants.py, DEFAULT_MAX_RETRIES



La prima chiamata di Claude Code

Dalla sessione che leggerete nell'esercizio che vedremo più tardi. La richiesta: *«La nota spese 1043 (cena a Milano del 14/07/2026, 52 euro) viene segnata fuori massimale, ma…»*. Cosa manda la harness alla prima chiamata, in quest'ordine, in caratteri:



la richiesta dell'utente: 309 caratteri, lo 0,4% ▲

27.979
token alla prima chiamata

22.674
già in cache: la parte uguale a quella di un'altra sessione

La cache: se l'inizio di una chiamata è identico a una già fatta, il fornitore non lo rielabora e lo fa pagare circa un decimo (slide 24). Tool e skill dipendono dalla macchina che ha registrato la sessione: su un'altra i numeri cambiano.

La sessione intera, senza tagli: `scripts/dataset/trace-claude-code-completa.md` (leggibile) · `trace-claude-code-completa.jsonl` (originale, stessa cartella)

La trace di Claude Code, per intero

GUARDIAMO INSIEME — il docente apre la trace, voi potete seguire

`scripts/dataset/trace-claude-code-completa.md · 3 minuti`

poi il docente riprende le slide

Nel file, dall'alto:

1	**I tool**	14 blocchi chiusi, ognuno con i suoi caratteri
2	**Il system prompt**	11 blocchi, per intero
3	**Il messaggio dell'utente**	i promemoria e, in fondo, la richiesta

In VS Code: tasto destro sul file, «Open Preview»; i blocchi chiusi si aprono con un clic. L'originale, `trace-claude-code-completa.jsonl`, è nella stessa cartella. Non è la trace dell'esercizio, che è più corta.

Il system prompt

1 ·	identità, sicurezza, **# Harness**	1.584
2 ·	__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__	
3 ·	stile del codice	94
4 ·	pronomi	357
5 ·	azioni irreversibili, dire com'è andata	520
6 ·	# Session-specific guidance	662
7 ·	# Memory (con il percorso)	2.206
8 ·	# Environment (i modelli)	598
9 ·	# Context management	280
10 ·	quando agire	277
11 ·	il budget di token della sessione: quanti ne restano	49

Sopra il marcatore: la parte fissa, uguale in tutte le sessioni. Sotto: quella che può cambiare da una sessione all'altra. A destra, i caratteri.

DAL BLOCCO «# HARNESS»

> Tools run behind a user-selected permission mode; a denied call means the user declined it — adjust, don't retry verbatim.
>
> I tool girano con una modalità di permessi scelta dall'utente; una chiamata negata vuol dire che l'utente ha detto no: cambia strada, non riprovare uguale.

> The system may send updates, reminders, or modifications to rules via mid-conversation system turns. These are system-controlled, unlike function results.
>
> Il sistema può mandare aggiornamenti, promemoria o modifiche alle regole con messaggi di sistema a metà conversazione. Sono controllati dal sistema, a differenza dei risultati dei tool.

Il system prompt spiega al modello **la harness in cui gira**: i permessi, i promemoria che arriveranno, il riassunto quando il contesto è pieno.

La sessione intera, senza tagli: `scripts/dataset/trace-claude-code-completa.md` (leggibile) · `trace-claude-code-completa.jsonl` (originale, stessa cartella)



Un tool, come lo vede il modello

```
// la definizione di Edit, com'è stata mandata (accorciata)
{
  "name": "Edit",
  "description": "Performs exact string replacement in a file.

    - You must Read the file in this conversation before editing, or the call will fail.
    - `old_string` must match the file exactly, including indentation, and be unique — the
edit fails otherwise. Strip the Read line prefix (line number + tab) before matching.
    - `replace_all: true` replaces every occurrence instead.",
  "input_schema": { "properties": {
    "file_path":   {"type": "string", "description": "The absolute path to the file to
modify"},
    "old_string":  {"type": "string", "description": "The text to replace"},
    "new_string":  {"type": "string", "description": "The text to replace it with …"},
    "replace_all": {"type": "boolean", "default": false} },
    "required": ["file_path", "old_string", "new_string"] }
}
```

«the call will fail»: la harness tiene l'elenco dei file letti. `Write` su un file mai letto: «`File has not been read yet.`»

«be unique»: un controllo nel codice. Quello di Claude Code non è pubblico: tra poco vediamo quello di riferimento di Anthropic.

Nella trace il modello legge con `cat` e la modifica passa: a quanto si vede, anche quella lettura conta. Ma Bash sconsiglia `cat`: la descrizione **orienta** il modello, non lo obbliga.

caratteri per tool

Edit	1.032
Bash	3.024
Artifact	27.849

Artifact, un tool per pubblicare pagine web, è quasi metà dei 58.714.

La sessione intera, senza tagli: `scripts/dataset/trace-claude-code-completa.md` (leggibile) · `trace-claude-code-completa.jsonl` (originale, stessa cartella)

I promemoria del primo messaggio

Prima della richiesta, dentro il messaggio dell'utente, la harness mette dei blocchi `<system-reminder>`. Accanto, i caratteri.

MESSAGGIO user

- `cartella, git sì/no, piattaforma, shell, cartella temporanea` · 584
- `il modello in uso` · 141
- `i nomi di 20 tool da caricare solo se servono` · 290
- `elenco di 6 tipi di agente e 16 skill` · 9.856
- `Codebase and user instructions are shown below. Be sure to adhere to these instructions. …`
  `Contents of …/CLAUDE.md (project instructions, checked into the codebase):`
  `# Rimborsi Valtesa · I test si lanciano con pytest. · … I file in `dati/` … non si modificano a mano.`
- `email dell'utente, stato di git` · 670
- `Today's date is 2026-09-28.`
- La nota spese 1043 (cena a Milano del 14/07/2026, 52 euro) viene segnata fuori massimale, ma… · 309

Il `CLAUDE.md` è il file di istruzioni del progetto: lo scrive chi ci lavora, la harness lo legge e lo inserisce così com'è.

I promemoria sono dopo il system prompt, non dentro: il system prompt resta uguale e resta in cache.

Una **skill**: istruzioni che la harness carica quando servono. Un **tipo di agente**: un altro agente, con un contesto suo (più avanti nel modulo).

`VOI` · la data era il problema di Modulo 3 · Lezione 7, e l'avete risolto con un tool.

La sessione intera, senza tagli: `scripts/dataset/trace-claude-code-completa.md` (leggibile) · `trace-claude-code-completa.jsonl` (originale, stessa cartella)



La prima chiamata di quattro harness

MINI-SWE-AGENT

~780

token: system 12, istruzioni con il compito 702, lo schema di bash 64

AIDER · /tokens, PRIMA DELLA RICHIESTA

2.839

```
$ 0.0008  1,926 system messages
$ 0.0000     95 repository map
$ 0.0000     82 CLAUDE.md (read-only)
$ 0.0001    193 tests/test_rimborsi.py
$ 0.0002    543 rimborsi.py
═══════════════
$ 0.0011  2,839 tokens total
```

CLAUDE CODE

27.979

token: 14 tool, system in 11 blocchi, promemoria, la richiesta

CODEX · un messaggio `developer` con permessi e sandbox, un messaggio `user` in due parti, gli `AGENTS.md` e `<environment_context>` (cartella, shell, data, fuso orario), poi la richiesta.

VOI · ISTRUZIONI e TOOLS: poche centinaia di token.

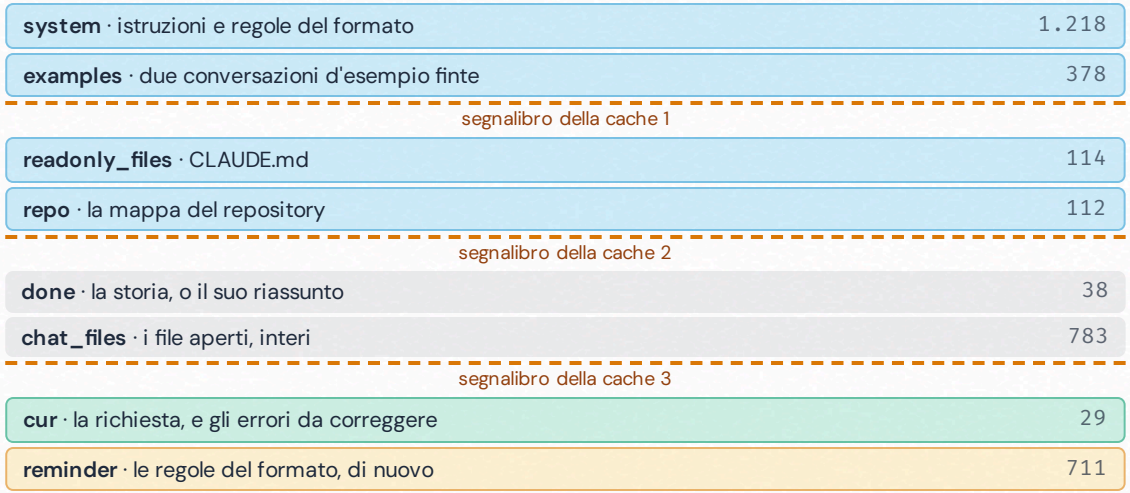
Codice: **mini-swe-agent** config/mini.yaml, system e istruzioni · lo schema di bash **Aider** commands.py, cmd_tokens() **Codex** test model_visible_layout, la prima richiesta · environment.rs, <environment_context>



L’ordine dei pezzi: Aider

Aider non tiene una lista che cresce: a ogni chiamata **rimonta il contesto da zero**, rileggendo i file dal disco.

```python
# aider/coders/chat_chunks.py
def all_messages(self):
    return (
        self.system
        + self.examples
        + self.readonly_files
        + self.repo
        + self.done
        + self.chat_files
        + self.cur
        + self.reminder
    )
```



Segnalibro: fin dove il fornitore tiene la cache (slide 24). Token misurati sul progetto Valtesa, con due file aperti, la richiesta e uno scambio di storia: circa 3.400 in tutto.

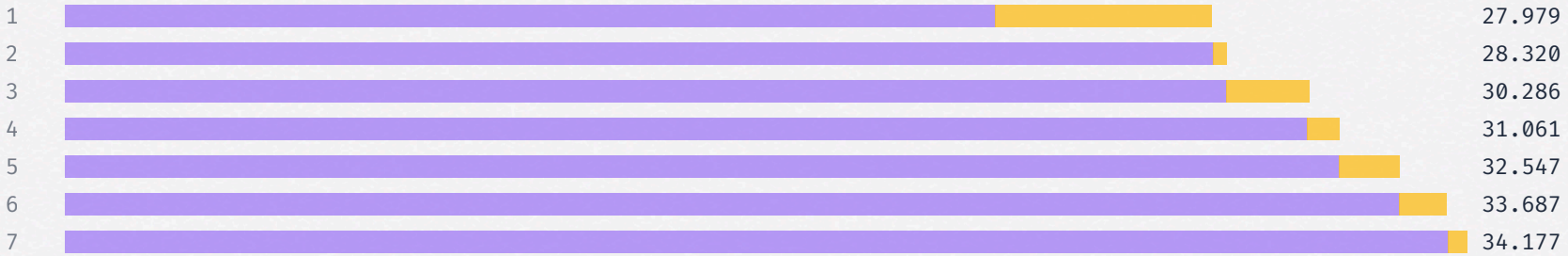
Dall'alto in basso: da quello che non cambia mai a quello che cambia a ogni chiamata. Le regole del formato delle modifiche vanno **due volte**, in cima e in fondo: circa 700 token per copia, più del 40% del totale.

Codice: **Aider** coders/chat_chunks.py, all_messages() · i segnalibri della cache · coders/base_coder.py, format_chat_chunks()

Perché le parti fisse vanno prima

Il fornitore tiene in cache l'**inizio identico** di una chiamata: prima i tool, poi il system, poi i messaggi. Rileggerlo costa un decimo (con Claude Opus 5.5, il modello della trace, un ventesimo). Per questo la harness non cambia mai l'inizio e **aggiunge solo in coda**.

LA TRACE · LE SETTE CHIAMATE DELLA PRIMA RICHIESTA · TOKEN IN INGRESSO



letti dalla cache non dalla cache In tutto il **94,7%** dalla cache.

Non dalla cache = «scritti in cache» (letti la prima volta e salvati per le chiamate dopo) + «nuovi» (letti e non salvati): nella trace li trovate separati.

Anthropic: la cache si chiede a ogni chiamata, con al massimo quattro segnalibri. **OpenAI**: automatica sopra i 1.024 token; con Luna, alla seconda chiamata di una conversazione da circa 3.200 token, 3.150 vengono dalla cache.

`VOI` · anche il vostro giro aggiunge solo in coda: quando l'inizio supera i 1.024 token, la cache c'è. `cost_usd` di aikit la ignora e conta tutto a prezzo pieno.

Quali tool, harness per harness

Harness	I tool che vede il modello
mini-swe-agent	`bash`
SWE-agent (paper)	`open`, `goto`, `scroll_up`, `scroll_down`, `create`, `edit`, `find_file`, `search_dir`, `search_file`, `submit`: ognuno uno script nel container
Aider	nessuno: il modello scrive le modifiche nel testo, la harness le estrae
OpenHands	`terminal`, `file_editor`, `task_tracker`, `finish`, `think` (+14 per il browser); a ogni tool aggiunge i campi `security_risk` e `summary`
Codex (GPT-5.5)	`exec_command`, `write_stdin`, `apply_patch`, `update_plan`, `view_image`, `web_search`, …
Claude Code	`Bash`, `Read`, `Edit`, `Write`, `Glob`, `Grep`, `Agent`, `Skill`, `ToolSearch`, … (14, più 20 da caricare)
Voi	`calcolatrice`, `data_oggi`, `query_db`, `prezzo_prodotto` (aikit); `query_db`, `cerca_documenti` (Modulo 3 · Lezione 9)

> Quasi tutte hanno una shell (in Aider i comandi li propone il modello, e partono solo se l'utente dice sì). Quasi tutte hanno anche un tool apposta per **modificare i file**: con la sola shell il modello sbaglia di più.

Codice: **mini-swe-agent** actions_toolcall.py, BASH_TOOL **SWE-agent** 07.yaml, i tool del paper **Aider** editblock_coder.py, get_edits() · base_coder.py, i comandi con un sì
OpenHands preset/default.py, i tool · tool.py, security_risk e summary **Codex** prompt_caching.rs, i tool con gpt-5.5



Modificare un file: i formati

File intero
Aider, formato whole

````
rimborsi.py
```
…tutto il file, riscritto…
```
````

SEARCH / REPLACE nel testo
Aider: il formato si chiama «diff»

```
<<<<<<< SEARCH
if r["voce"] == voce:
=======
if r["voce"] == voce and …:
>>>>>>> REPLACE
```

Sostituisci questo testo
Claude Code `Edit`, OpenHands, Anthropic

```
Edit(file_path=…,
  old_string="…",
  new_string="…")
```

Una patch
Codex, `apply_patch`

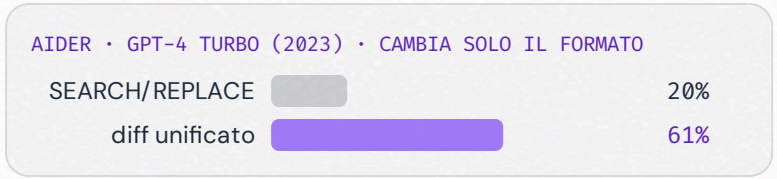
```
*** Begin Patch
*** Update File: rimborsi.py
- if r["voce"] == voce:
+ if r["voce"] == voce and …:
*** End Patch
```

Righe da-a
SWE-agent, `edit`

```
edit 41:43
…le righe nuove…
end_of_edit
```

Dalla shell
mini-swe-agent: il prompt dà gli esempi

```
sed -i 's/old_string/new_string/g'
filename.py
```



Il diff unificato è un settimo formato: righe tolte con `-`, aggiunte con `+`, come `git diff`. Il migliore dipende dal modello: Aider lo sceglie modello per modello. I fornitori usano quello su cui il modello è **addestrato**: la patch per GPT, «sostituisci questo testo» per Claude.

Codice: **mini-swe-agent** mini.yaml, gli esempi con sed **SWE-agent** windowed_edit_linting, edit **Aider** editblock_prompts.py, SEARCH/REPLACE · model-settings.yml
OpenHands file_editor/definition.py **Codex** apply_patch.lark, la grammatica · apply_patch_spec.rs

str_replace, il codice

```python
# il tool di riferimento di Anthropic, edit.py (tagliato)
def str_replace(self, path, old_str, new_str):
    file_content = self.read_file(path)
    occurrences = file_content.count(old_str)
    if occurrences == 0:
        raise ToolError(
            f"No replacement was performed, old_str `{old_str}` "
            f"did not appear verbatim in {path}.")
    elif occurrences > 1:
        lines = [idx + 1 for idx, line in enumerate(file_content.split("\n"))
                 if old_str in line]
        raise ToolError(
            f"No replacement was performed. Multiple occurrences of "
            f"old_str `{old_str}` in lines {lines}. Please ensure it is unique")
    new_file_content = file_content.replace(old_str, new_str)
    self.write_file(path, new_file_content)
    ...    # uno spezzone: 4 righe prima, la modifica, 4 righe dopo
```

Il modello non scrive il file: dice **cosa sostituire**. La harness accetta solo se il testo compare **una volta**.

SE VA BENE, AL MODELLO TORNA (SU UN FILE D'ESEMPIO)

```
The file …/prezzi.py has been edited. Here's the
result of running `cat -n` on a snippet of
…/prezzi.py:
    6  def prezzo(codice):
    7      if codice not in PREZZI:
    8          return None
    …
Review the changes and make sure they are as expected.
Edit the file again if necessary.
```

Con due occorrenze: «… `in lines [7, 10]. Please ensure it is unique`». Il messaggio dice al modello dove guardare.

Codice: **tool di riferimento di Anthropic** tools/edit.py, str_replace()



Errori scritti per il modello

SWE-AGENT · IL LINTER (CONTROLLA LA SINTASSI) RIFIUTA LA MODIFICA

```
Your proposed edit has introduced new syntax error(s). Please read
this error message carefully and then retry editing the file.

ERRORS:
{errors}

This is how your edit would have looked if applied
{window_applied}

This is the original code before your edit
{window_original}

Your changes have NOT been applied. Please fix your edit command
and try again.
DO NOT re-run the same failed edit command. Running it again will
lead to the same error.
```

Senza questo controllo: 3 punti in meno (slide 7).

AIDER · IL BLOCCO SEARCH NON COMBACIA

```
# 1 SEARCH/REPLACE block failed to match!

## SearchReplaceNoExactMatch: This SEARCH block failed to exactly
match lines in rimborsi.py
<<<<<<< SEARCH
    raise KeyError(f"nessun massimale per {voce} e {citta}")
…
Did you mean to match some of these actual lines from rimborsi.py?

    raise KeyError(f"nessun massimale per {voce} a {citta}")

The SEARCH section must exactly match an existing block of lines
including all white space, comments, indentation, docstrings, etc
```

Il messaggio d'errore diventa il messaggio successivo dell'utente. Al massimo 3 volte.

VOI · l'errore SQL di Modulo 3 · Lezione 8, rimandato al modello come testo: la stessa idea. Qui il messaggio dice anche **cosa fare**.

Codice: **SWE-agent** windowed_edit_linting/bin/edit, messaggio del linter **Aider** coders/editblock_coder.py, il messaggio d'errore · base_coder.py, max_reflections



La shell: nuova ogni volta o sempre la stessa

UNA BASH APERTA · COME CAPIRE CHE IL COMANDO È FINITO

```python
# documentazione Anthropic, il tool bash
def execute_command(self, command):
    sentinel = f"__CLAUDE_BASH_DONE_{uuid.uuid4().hex}__"
    self.process.stdin.write(f"{command}\necho {sentinel}\n")
    self.process.stdin.flush()

    output = []
    for line in self.process.stdout:
        if sentinel in line:     # il comando ha finito
            break
        output.append(line)
    return "".join(output)
```

Dopo il comando, echo di una parola che non può comparire per caso: si legge fino a lì.

mini-swe-agent · un processo nuovo a ogni comando: non resta niente, e il prompt lo dice.

OpenHands · una sessione tmux, un terminale che resta aperto; la riga che la shell stampa prima di ogni comando è un JSON con exit code e cartella.

Terminus 2 · il modello manda tasti a tmux e decide lui quanti secondi aspettare.

Claude Code · la cartella resta tra un comando e l'altro, le variabili no: lo dice la descrizione di Bash. Nella trace troverete un'eccezione.

Codice: **mini-swe-agent** local.py, _run() · mini.yaml, la subshell nuova **OpenHands** terminal/metadata.py, il PS1 in JSON · tmux_terminal.py **Anthropic** documentazione del bash tool, la sentinella **Terminus 2** terminus-json-plain.txt · terminus_2.py, _execute_commands()



Il comando che non finisce

Un server che parte, un `input()` che aspetta, un test bloccato: se la harness aspetta per sempre, il giro si ferma.

OPENHANDS · DOPO 30 SECONDI SENZA OUTPUT NUOVO, LA PAROLA TORNA AL MODELLO

```
[The command has no new output after 30 seconds. You may wait longer to see additional output by sending empty command '', send other commands to interact with the current process, send keys ("C-c", "C-z", "C-d") to interrupt/kill the previous command before sending your new command, or use the timeout parameter in terminal for future commands.]
```

mini-swe-agent · dopo 30 s uccide il comando: «`Command '…' timed out after 30 seconds`».

Tool di riferimento di Anthropic · dopo 120 s la bash è bloccata: il modello deve chiedere `restart`.

Claude Code · 2 minuti di default, fino a 10; oppure il comando va in background.

SWE-agent · i comandi interattivi non li lancia: «`Operation 'vim rimborsi.py' is not supported by this environment.`»

Codice: **mini-swe-agent** local.py, _run() e il timeout · execute() **SWE-agent** sweagent/tools/tools.py, comandi bloccati **OpenHands** terminal/constants.py, 30 s · terminal_session.py, il messaggio **tool di riferimento di Anthropic** tools/bash.py, timeout di 120 s · restart



Output troppo lunghi

Sul progetto dell'esercizio `python rimborsi.py` stampa **108.707 caratteri**; l'ultima riga è il totale. Cosa arriva al modello:



Anthropic taglia a 16.000 caratteri quando l'editor apre un file, e in coda mette questa nota:

```
<response clipped><NOTE>To save on context only part of this file has been shown to you. You should retry this tool after you have searched inside the file with `grep -n` in order to find the line numbers of what you are looking for.</NOTE>
```

Claude Code salva gli output troppo lunghi su un file e manda al modello un'anteprima di circa 2.200 caratteri, con il percorso.

`VOI` · `LIMITE = 50` righe in `query_db`, e la riga che lo dice.

Codice: **mini-swe-agent** mini.yaml, observation_template **SWE-agent** sweagent/agent/agents.py, max_observation_length **OpenHands** terminal/constants.py, 30.000 · utils/truncate.py **tool di riferimento di Anthropic** tools/bash.py, run() · edit.py, _make_output() · run.py, maybe_truncate()

Quando la harness scrive nella conversazione

all'avvio	a ogni chiamata	dopo ogni tool	su evento	ogni N passi	prima di chiudere	al limite	contesto pieno
system, tool, istruzioni del progetto, ambiente, data tutte	le regole del formato, ripetute in fondo Aider	il risultato in un formato suo; dove sei mini, SWE-agent	formato sbagliato, linter, stesso errore 3 volte mini, SWE-agent, OpenHands	il piano, rifatto smolagents	rileggi le tue modifiche SWE-agent	dai comunque una risposta smolagents	il riassunto al posto della storia OpenHands, Claude Code, Codex · ne parliamo dopo l'esercizio

Anche Claude Code ne scrive: nella trace dell'esercizio li trovate tra una chiamata e l'altra, e a volte dentro un risultato.

Codice: **Aider** base_coder.py, le regole in fondo



Dopo ogni passo: file aperto, cartella, risultato

```python
# SWE-agent, tools/windowed/bin/_state (tagliato)
# gira dopo ogni azione, lanciato dalla harness
def main():
    state_path = Path("/root/state.json")
    state = json.loads(state_path.read_text()) \
            if state_path.exists() else {}
    current_file = registry.get("CURRENT_FILE")
    open_file = "n/a" if not current_file \
                else str(Path(current_file).resolve())
    state["open_file"] = open_file
    state["working_dir"] = os.getcwd()
    state_path.write_text(json.dumps(state))
```

DOPO open: LA FINESTRA DI 100 RIGHE DELLA SLIDE 7 (RICOSTRUITA DAL CODICE)

```
[File: dati/note_spese.csv (1501 lines total)]
(27 more lines above)
28:1026,Chiara Lombardi,hotel,Milano,2026-08-04,109.28
…
45:1043,Luca Ferri,cena,Milano,2026-07-14,52.00
…
(1374 more lines below)
(Open file: /valtesa-rimborsi/dati/note_spese.csv)
(Current directory: /valtesa-rimborsi)
bash-$
```

Aider, dopo ogni modifica applicata: un commit git, e nella storia «`I committed the changes with git hash {hash} & commit msg: {message}`».

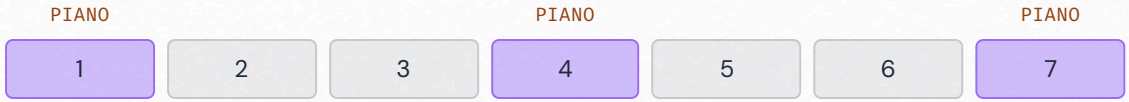
smolagents: ogni risultato torna come «`Observation: …`»; un errore come «`Error: … Now let's retry: take care not to repeat previous errors!`».

Codice: **SWE-agent** tools/windowed/bin/_state · windowed_file.py, la finestra · 07.yaml, next_step_template **Aider** base_coder.py, il commit dopo la modifica · base_prompts.py, il messaggio **smolagents** memory.py, Observation ed Error



Ogni N passi: il piano

```python
# smolagents, agents.py: all'inizio di ogni passo
if self.planning_interval is not None and (
    self.step_number == 1
    or (self.step_number - 1) % self.planning_interval == 0
):
    ...    # una chiamata di pianificazione, poi il passo normale
```



con `planning_interval=3`

La chiamata di piano ha un prompt suo: i fatti noti, i fatti da cercare, il piano.

Il piano rientra nella storia come messaggio del modello, e subito dopo la harness scrive:

```
Now proceed and carry out this plan.
```

Dopo molti passi il modello perde di vista il compito: il piano glielo rimette davanti.

Codice: **smolagents** agents.py, planning_interval · prompts/code_agent.yaml, il prompt del piano · memory.py, il piano nella storia



Prima di chiudere e al limite

SWE-AGENT · AL PRIMO submit NON CONSEGNA

```
Thank you for your work on this issue. Please carefully follow the
steps below to help review your changes.

1. If you made any changes to your code after running the
reproduction script, please run the reproduction script again.
  If the reproduction script is failing, please revisit your
changes and make sure they are correct.
2. Remove your reproduction script (if you haven't done so
already).
…
4. Run the submit command again to confirm.

Here is a list of all of your changes:

<diff>
{{diff}}
</diff>
```

Consegna al secondo `submit`. Un giro in più, imposto dal codice.

SMOLAGENTS · 20 PASSI SENZA RISPOSTA

```
An agent tried to answer a user query but it got stuck and failed
to do so. You are tasked with providing an answer instead. Here is
the agent's memory:
…
Based on the above, please provide an answer to the following user
task:
{{task}}
```

Un'ultima chiamata, senza tool: una risposta arriva comunque.

mini-swe-agent al limite: niente, la corsa finisce.

Agents SDK: un'eccezione (Modulo 3 · Lezione 10).

Codice: **mini-swe-agent** default.py, query() al limite **SWE-agent** config/default.yaml, messaggio di revisione · bin/submit, due volte **smolagents** agents.py, provide_final_answer() · code_agent.yaml, final_answer **Agents SDK 0.22.3** run.py, MaxTurnsExceeded



Perché nei messaggi e non nel system prompt

1. **La cache.** Cambiare l'inizio la rompe. Codex, alla seconda richiesta, non riscrive i permessi e l'ambiente: li **aggiunge in coda**, anche se sono cambiati.

2. **La vicinanza.** Le regole importanti si ripetono in fondo, dove il modello le legge per ultime: Aider manda le regole del formato due volte.

3. **Il modello è avvisato.** Sa che arriveranno messaggi della harness, e che valgono più dei risultati dei tool.

```
«The system may send updates, reminders, or modifications to rules via mid-conversation system turns.
These are system-controlled, unlike function results.»
```

Il sistema può mandare aggiornamenti, promemoria o modifiche alle regole con messaggi di sistema a metà conversazione. Sono controllati dal sistema, a differenza dei risultati dei tool.

il system prompt di Claude Code, blocco «# Harness»

Codice: **Aider** coders/base_coder.py, le regole in cima · e in fondo **Codex** test model_visible_layout, la seconda richiesta



Esercizio: leggere una trace

FATE VOI — ognuno da solo, niente codice

`scripts/dataset/trace-claude-code.md` · `scripts/working/scheda.md` · 20 minuti

poi 5' di rientro:
il docente riprende le slide

La trace è il file che Claude Code salva per ogni sessione; questa è sul progetto Valtesa (correggere il bug, poi esportare un file Excel). In testa alla trace c'è la prima chiamata: il system prompt blocco per blocco, i tool, i promemoria. Per ogni domanda scrivete nella scheda la risposta, il pezzo della harness e dove ne avete scritto la versione di base (lezione e nome nel codice).

1 · Da cosa si capisce che il giro è finito? Quante chiamate per la seconda richiesta?

2 · Che messaggi aggiunge la harness tra una chiamata e l'altra, e dentro i risultati?

3 · Quando il modello chiede due `Edit` insieme: con quali parametri? E se il testo comparisse due volte?

4 · Come conta le note che cambiano esito senza leggere gli output interi?

5 · Cosa va storto nella seconda richiesta, e cosa fa il modello dopo?

6 · All'ultima chiamata della seconda richiesta, quanti token dalla cache, e perché?

`FATTO QUANDO` · le sei domande della scheda hanno risposta, pezzo e «Voi» (la tabella delle lezioni è in fondo alla scheda)

La sessione intera, senza tagli: `scripts/dataset/trace-claude-code-completa.md` (leggibile) · `trace-claude-code-completa.jsonl` (originale, stessa cartella)

Rientro: la scheda

	Risposta	Pezzo	Voi
1	il modello risponde senza chiedere tool (chiamata 7, poi 5); la seconda richiesta usa 5 chiamate	il giro	Modulo 3 · Lezioni 7-9: `if not chiamate: return` in `chiedi()`
2	il budget di token dopo ogni richiesta e dopo ogni risultato; la lista dei compiti, vuota; dentro un risultato «Shell cwd was reset…»	il contesto (anche i limiti)	Modulo 3 · Lezione 3: i due messaggi di `riassumi()` ; Modulo 3 · Lezione 8: «(mostrate solo le prime 50 righe)»
3	richiesta 1, chiamata 4: `file_path` , `old_string` , `new_string` , `replace_all` ; con due occorrenze la modifica fallisce	i tool	Modulo 3 · Lezione 7: nome, descrizione, parametri
4	il modello, non la harness, salva i due output su file (chiamate 3 e 5), li confronta con `diff` e conta: 19	gli output	Modulo 3 · Lezione 8: `LIMITE = 50`
5	`ModuleNotFoundError: openpyxl` ; installa la libreria in una cartella temporanea, non nel progetto	gli errori	Modulo 3 · Lezione 8: l'errore SQL come testo
6	37.694 dalla cache, 2.107 no (2.105 scritti in cache, 2 nuovi): la harness ha solo aggiunto in coda	il contesto	la cache la fa il fornitore; anche `chiedi()` aggiunge in coda



Quando il contesto si riempie: i rimedi

Più contesto non vuol dire risposte migliori: con input più lunghi i modelli peggiorano, anche su compiti semplici (misurato su 18 modelli, 2025). E ogni token si paga a ogni chiamata. I rimedi, dal più semplice:

1
Tagliare alla fonte
gli output lunghi: testa e coda, o su file
già visto

2
Togliere i vecchi output
al loro posto una riga
SWE-agent, Anthropic

3
Riassumere
un riassunto al posto della storia
OpenHands, Claude Code, Codex, Terminus 2

4
Scegliere cosa mettere
non tutto il repository: una mappa; non tutti i tool: i nomi
Aider, Claude Code

5
Un altro agente
con un contesto suo, che torna con un rapporto
lo vedrete più avanti nel modulo

> Il 2 e il 3 cambiano la parte già mandata, e la **cache** di quella parte si perde: per questo spesso scattano sopra una soglia (100.000 token nell'API di Anthropic, il 90% della finestra in Codex). SWE-agent toglie i vecchi output a ogni passo.

Codice: **SWE-agent** history_processors.py, LastNObservations **Codex** openai_models.rs, la soglia al 90% **Terminus 2** terminus_2.py, _summarize()



Togliere i vecchi output

```python
# SWE-agent, history_processors.py, LastNObservations (tagliato)
observation_indices = [idx for idx, entry in enumerate(history)
                       if entry.get("message_type") == "observation"]
last_removed_idx = max(0, len(observation_indices) - self.n)   # n = 5
omit = observation_indices[1:last_removed_idx]   # la prima è il compito
for idx, entry in enumerate(history):
    if idx not in omit:
        new_history.append(entry)
    else:
        data = entry.copy()
        n_lines = len(data["content"].splitlines())
        data["content"] = f"Old environment output: ({n_lines} lines omitted)"
        new_history.append(data)
```

Si cambia solo quello che vede il modello: la storia completa resta nella traccia, su disco.

COSA VEDE IL MODELLO

- il compito
- `Old environment output: (37 lines omitted)`
- `Old environment output: (212 lines omitted)`
- `Old environment output: (8 lines omitted)`
- output 1 di 5, intero
- …
- output 5 di 5, intero

Nel paper: **18,0%** con gli ultimi 5, **15,0%** con tutta la storia.

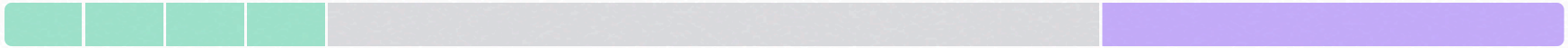
Anthropic fa la stessa cosa sul server (*context editing*): sopra 100.000 token toglie i risultati dei tool più vecchi, tiene gli ultimi 3 e lascia al loro posto un segnaposto.

Codice: **SWE-agent** sweagent/agent/history_processors.py, LastNObservations **Anthropic** documentazione del context editing



Riassumere: il condensatore di OpenHands

OpenHands salva ogni evento su disco (azioni, risultati, messaggi) e a ogni giro costruisce la **vista** da mandare al modello.



i primi 4 eventi: si tengono — quelli di mezzo: riassunti da una chiamata apposita — i più recenti: si tengono

Quando: oltre **80 eventi**, oltre il limite di token, o quando il fornitore risponde «`context window exceeded`». Il riassunto rientra come **messaggio dell'utente**.

il prompt del riassunto (tagliato)

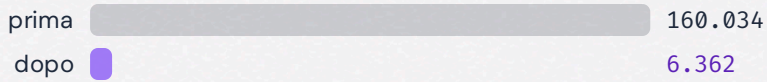
```
You are maintaining a context-aware state summary for an interactive agent.
You will be given a list of events corresponding to actions taken by the agent, which will include
previous summaries.
…
USER_CONTEXT: (Preserve essential user requirements, goals, and clarifications in concise form)
TASK_TRACKING: {Active tasks, their IDs and statuses - PRESERVE TASK IDs}
COMPLETED: (Tasks completed so far, with brief results)
PENDING: (Tasks that still need to be done)
CURRENT_STATE: (Current variables, data structures, or relevant state)
```

`VOI` · `riassumi()` di Modulo 3 · Lezione 3: la stessa idea, fatta a mano. Qui il prompt dice **cosa non perdere**.

Codice: **OpenHands** condenser/llm_summarizing_condenser.py, 80 e 4 · quando scatta · prompts/summarizing_system.j2

Riassumere: cosa legge il modello dopo

CLAUDE CODE · LA COMPATTAZIONE, CIOÈ IL RIASSUNTO, CHIESTA A MANO IN UN'ALTRA SESSIONE



in 116 secondi. Una automatica, in un'altra sessione con una finestra da 1 milione di token, è scattata a 967.146.

```
This session is being continued from a previous conversation
that ran out of context. The summary below covers the earlier
portion of the conversation.
```

CODEX · AL 90% DI 272.000 TOKEN

Con i modelli OpenAI il riassunto lo fa il server e torna cifrato. Con gli altri lo scrive il modello, e Codex lo presenta così:

```
Another language model started to solve this problem and
produced a summary of its thinking process. You also have
access to the state of the tools that were used by that
language model. Use this to build on the work that has already
been done and avoid duplicating work. …
```

Dopo il riassunto il modello non sa di aver lavorato lui: legge che è stato un altro modello.

TERMINUS 2 · SOTTO 8.000 TOKEN LIBERI, IN TRE PASSI

il modello scrive un riassunto per chi continua → un'istanza nuova, senza storia, lo legge e fa almeno 5 domande → il modello con la storia risponde; si riparte da riassunto, domande e risposte

Codice: **Codex** compact/summary_prefix.md · compact.rs · openai_models.rs, il 90% **Terminus 2** terminus_2.py, soglia di 8.000 token · _summarize() in tre passi



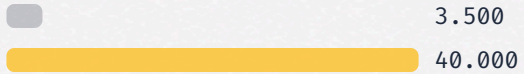
Scegliere cosa mettere

Aider non manda il repository intero: una **repo map**, le definizioni più importanti di ogni file. Le estrae con tree-sitter (un parser per molti linguaggi), le ordina con PageRank (l'algoritmo con cui Google ordinava le pagine) sui riferimenti tra file, e si ferma a un budget: **1/8 della finestra**, tra 1.024 e 4.096 token.

una repo map, dalla documentazione di Aider (tagliata)

```
aider/coders/base_coder.py:
⋮ ...
│class Coder:
│    abs_fnames = None
⋮ ...
│    def run(self, with_message=None):
⋮ ...
aider/commands.py:
⋮ ...
│class Commands:
│    def run(self, inp):
⋮ ...
```

I file interi entrano quando il modello li nomina e l'utente dice sì. Sul progetto Valtesa il modello ha nominato `note_spese.csv`, e la chiamata dopo è passata da



token.

Claude Code fa lo stesso con i tool: di 20 manda solo il nome, e il modello li carica con `ToolSearch` quando servono.

Codice: **Aider** models.py, il budget della repo map · repomap.py, PageRank · base_coder.py, i file nominati dal modello

Errori dell'API: si riprova

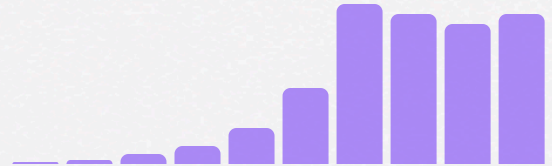
```
# openai 3.19.2, _base_client.py (tagliato): la libreria che usate
# Honor server-directed delays up to two minutes.
retry_after = self._parse_retry_after_header(response_headers)
if retry_after is not None and 0 < retry_after ⩽ MAX_RETRY_AFTER_DELAY:
    return retry_after

nb_retries = min(max_retries - remaining_retries, 1000)
# Apply exponential backoff, but not more than the max.
sleep_seconds = min(INITIAL_RETRY_DELAY * pow(2.0, nb_retries),
                    MAX_RETRY_DELAY)                 # 0,5 s · 8 s
# Reduce the calculated timeout by a random range between 0-25%
jitter = 1 - 0.25 * random()
timeout = sleep_seconds * jitter
```

Si riprova su 408 (tempo scaduto), 409 (conflitto), 429 (troppe richieste), 5xx (errore del server), timeout ed errori di connessione. Mai su 400 (richiesta sbagliata) o 401 (chiave sbagliata): riprovare non servirebbe.

VOI · `chiedi()` crea `client = OpenAI()`: 2 tentativi in più (circa 0,5 e 1 s) e 10 minuti di timeout. Il client di Modulo 2 · Lezione 3: 3 tentativi in più, 30 s.

CLAUDE CODE · UN'ALTRA SESSIONE VERA · ATTESE IN SECONDI



10 tentativi, poi nella storia un messaggio «Request `timed out`» che il modello non ha scritto.

Codice: **client openai 3.19.2** _constants.py, i default · _base_client.py, _calculate_retry_timeout() · _should_retry()

Quando il modello gira a vuoto

OpenHands, `StuckDetector`: prima di ogni passo guarda gli ultimi 20 eventi e cerca quattro schemi.



stessa azione, stesso risultato, **4** volte



stessa azione, stesso errore, **3** volte



3 messaggi di fila senza fare niente



due azioni che si alternano, **6** volte

ALLO STESSO ERRORE RIPETUTO, PRIMA SCRIVE AL MODELLO

```
You've called `{tool_name}` with the same arguments 3 times in a row and gotten the same error each time: {error}. Repeating the exact same call again will not work — review the error message and either correct the arguments or try a different approach.
```

Se continua, ferma il giro e aspetta una persona. mini-swe-agent: tre errori di formato di fila, e la corsa finisce. Un tetto come `MAX_CHIAMATE` da solo non basta: un agente vero fa decine di chiamate, e un giro a vuoto va riconosciuto prima.

Codice: **mini-swe-agent** agents/default.py, errori di formato di fila · max_consecutive_format_errors = 3 **OpenHands** conversation/types.py, le soglie · stuck_detector.py, is_stuck() · il messaggio al modello



Come finisce il giro

	Come finisce	Al limite
Voi · `chiedi()`	una risposta senza tool	`MAX_CHIAMATE` : un messaggio all'utente
Agents SDK	la risposta finale	`max_turns` : un'eccezione
mini-swe-agent	`echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`	passi o costo: `exit` , al modello niente
SWE-agent	`submit` , due volte	costo: consegna le modifiche fatte fin lì
smolagents	il tool `final_answer`	20 passi: la risposta forzata
OpenHands	il tool `finish` , o una risposta senza tool	500 iterazioni: un errore
Claude Code	una risposta senza tool	nessun tetto di chiamate di default

> Due famiglie: finisce quando il modello **smette di chiedere tool**, o quando **chiama il tool della fine**. La seconda obbliga il modello a dire che ha finito.

Codice: **mini-swe-agent** local.py, _check_finished() · default.py, query() al limite **SWE-agent** agents.py, autosubmission · bin/submit, due volte **OpenHands** agent.py, finish · local_conversation.py, 500 **smolagents** agents.py, _run_stream() · default_tools.py, FinalAnswerTool **Agents SDK 0.22.3** run.py, MaxTurnsExceeded · turn_resolution.py



Chi decide cosa può fare l'agente

1 · NIENTE PERMESSI, UN RECINTO FUORI

mini-swe-agent esegue tutto (il comando `mini`, di default, chiede conferma prima di ogni comando); per isolarlo si cambia `subprocess` con `docker exec`. **smolagents** esegue il codice del modello in un interprete suo, che vieta `os`, `subprocess`, `socket`…

2 · MODALITÀ E REGOLE

Claude Code, nella trace, era in `acceptEdits`: le modifiche ai file passano senza chiedere. Regole per tool e per comando. Se l'utente dice no, il system prompt dice al modello di cambiare strada.

3 · LA SANDBOX: UN RECINTO DEL SISTEMA OPERATIVO

Codex lancia i comandi in un profilo che parte da «nega tutto»: Seatbelt su macOS, bubblewrap e seccomp su Linux. Scrittura solo nella cartella di lavoro, rete spenta.

4 · IL MODELLO DICHIARA IL RISCHIO

OpenHands aggiunge a ogni tool `security_risk`: LOW, MEDIUM, HIGH. Con la policy che chiede conferma, HIGH e «non dichiarato» aspettano una persona. Con le impostazioni di default nessuno lo legge.

`VOI` · il database aperto in sola lettura, `mode=ro`: è il modo 1, un limite messo fuori dal modello.

Codice: **mini-swe-agent** local.py, subprocess · docker.py, docker exec **OpenHands** agent.py, security_risk · confirmation_policy.py, ConfirmRisky · state.py, i default
smolagents local_python_executor.py · utils.py, import autorizzati **Codex** seatbelt_base_policy.sbpl · bwrap.rs · landlock.rs, seccomp



La trifecta letale

Il rischio da ricordare (Simon Willison): con tutti e tre, un contenuto ostile può far uscire i dati privati.

- dati privati
- contenuti non fidati
- un modo per mandare dati fuori

Per esempio: un agente che legge la vostra posta, apre una pagina web con istruzioni nascoste e può mandare email.

Il modello non distingue da solo un'istruzione vostra da una scritta dentro una pagina: per questo i limiti li mette la harness. La sicurezza: Modulo 5 · Lezione 8.



Riepilogo: voi e una harness vera

Pezzo	Voi	Una harness vera
Il giro	`chiedi()`	la fine dichiarata con una stringa o un tool, una revisione prima di chiudere
I limiti	`MAX_CHIAMATE`	passi, costo, tempo; i giri a vuoto riconosciuti
Gli errori	l'errore come testo in `query_db` ; 2 retry	messaggi scritti per il modello (linter, SEARCH, `str_replace`); retry con attese fino a decine di secondi
Il contesto	`ISTRUZIONI` , la storia, il riassunto	28.000 token prima di cominciare, promemoria in coda, vecchi output tolti, riassunti, repo map
Gli output	`LIMITE = 50`	testa e coda, una nota per il modello, il resto su file
I tool	3–4 tool vostri	shell, editor con controllo di unicità, formati su cui il modello è addestrato
I permessi	`mode=ro`	modalità, regole, sandbox, rischio dichiarato
La traccia	la lista dei tool usati, il costo	la storia salvata su file a ogni passo, una trace per sessione



Perché non scriverne una da zero

VOI · chiedi()

47

righe

MINI-SWE-AGENT

190

righe l'agente, 5.080 il pacchetto

AGENTS SDK

46.161

righe nel nucleo, senza test

CODEX

~139.000

righe nel nucleo; 429 commit in una settimana

1 **Harness e modello vanno insieme.** Il formato delle modifiche, i prompt, le soglie sono scelti per il modello: «Many agent scaffolds have been engineered to accommodate the tendencies of certain models» (gli autori di Terminal-Bench).

molte harness sono progettate per adattarsi alle tendenze di certi modelli

2 **I modelli cambiano ogni pochi mesi**, e una parte della harness va tolta: «every component in a harness encodes an assumption about what the model can't do on its own» (Anthropic, 2026).

ogni pezzo di una harness contiene un'ipotesi su cosa il modello non sa fare da solo

Codice: **mini-swe-agent** agents/default.py · il pacchetto **Agents SDK 0.22.3** il pacchetto



aikit o l'Agents SDK?

AIKIT

i pezzi: loader, pulizia, chunk, embedding, vector DB, RAG, `query_db`

e un giro: `chiedi()` (Modulo 3 · Lezioni 7-9; in aikit, la versione di Modulo 3 · Lezione 8)

→

i pezzi diventano i tool

LA HARNESS

il giro, i limiti, gli errori, il contesto, la traccia

Per lavorare, questo lo fa una harness esistente: l'Agents SDK, a cui date voi tool e istruzioni; poi i coding agent, che arrivano completi.

I pezzi restano vostri. Il giro di `aikit` resta per **capire** come funziona: è quello che avete fatto in Modulo 3 · Lezioni 7-9.

Mantenere una harness **costa**, e non è il lavoro che vi serve fare.



I coding agent e il setup per la prossima lezione

Agenti che lavorano su una cartella di codice: leggono e scrivono file, lanciano comandi.

Claude Code
Anthropic, sui modelli Claude

Codex
OpenAI, sui modelli GPT

Sviluppate da chi addestra il modello: permessi, sessioni lunghe, progetti veri. Sui benchmark non sono sempre prime (Claude Code 58,0%, Terminus 2 62,9%): i benchmark misurano compiti chiusi, e alcune harness sono tarate proprio su quelli.

MENTRE LI USATE, GUARDATE

- la trace: Claude Code la salva in `~/.claude/projects/`, Codex in `~/.codex/sessions/`
- cosa riceve il modello, e i promemoria
- i permessi che chiedono
- quando compattano
- quanto contesto consumano

PRIMA DELLA PROSSIMA LEZIONE

1. Installate il coding agent e fate il login con il vostro account: Claude Code con Claude Pro, oppure Codex con ChatGPT Plus. L'abbonamento è a vostro carico.
2. Fate `git pull` della repo del corso: il progetto Valtesa, con l'assistente di Modulo 3 · Lezione 12, è in `modulo4-agenti/l02_coding-agent/scripts/`.
3. In quella cartella copiate `.env.example` in `.env` e scriveteci la chiave OpenAI nuova del corso.
4. Chiedete al coding agent di creare il venv del modulo in `modulo4-agenti/.venv` e di lanciare `python indicizza.py` e `pytest`: deve uscire `11 passed`.
5. Entro il giorno prima della lezione, scrivete nel canale del corso la versione del coding agent e l'ultima riga di `pytest`.

I passi completi sono nel messaggio sul canale del corso.

Codice: **Codex** rollout/src/recorder.rs, ~/.codex/sessions



Grazie

prossima lezione: Modulo 4 · Lezione 2

